

Desarrollo del Proceso de Canje de Entradas

Actividad N°: 23 **Fecha:** 01/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar y documentar el proceso completo de canje de boletos (individual y masivo), integrando validación de datos, actualización de estado, auditoría y notificación en tiempo real.

2. Secuencia completa del canje individual (**canjeTicket**)

```

POST /api/tickets/:id/canje { quienRetira, parentesco?, quienOtro?, celular }
|
▼
Validar: quienRetira y celular obligatorios
|
▼
Si quienRetira = "Otro": validar parentesco y quienOtro obligatorios
|
▼
Validar que quienRetira ∈ {Titular, Titular Compra, Otro}
|
▼
Buscar ticket por 'Ticket ID' → 404 si no existe
|
▼
¿ticket.canjeado === true? —Sí→ 400 "Este ticket ya fue canjeado"
| No
▼
Actualizar: canjeado=true, fechaCanje=now, usuarioCanje, puntoCanje,
            quienRetira, celular (+ parentesco/quienOtro si aplica)
|
▼
ticket.save()
|
▼
Crear AuditLog { tipo: 'canje', ticketId, transactionId, detalles, ip }
            (si falla el log, NO se revierte el canje – se registra el error aparte)
|
▼
Emitir Socket.IO 'ticket-updated' a sala punto-venta-{id} y staff-{puntoTrabajo}
|
▼
200 { ticket actualizado }

```

3. Secuencia del canje masivo (**bulkCanjeTickets**)

```

POST /api/tickets/bulk-canje { ticketIds[], canjeData }
    |
    ▼
Validar ticketIds no vacío + campos obligatorios de canjeData
    |
    ▼
Buscar TODOS los tickets solicitados (canjeados y no canjeados)
    |
    ▼
Separar en dos grupos: ticketsToRedeem (pendientes) / alreadyRedeemed (ya
canjeados)
    |
    ▼
¿ticketsToRedeem vacío? —Sí→ 400 "Todos ya fueron canjeados"
    | No
    ▼
Construir un único updateData común (mismos datos de retiro para todos)
    |
    ▼
bulkWrite(): un updateOne por cada ticket pendiente, en una sola operación a
MongoDB
    |
    ▼
Insertar un AuditLog por cada ticket recién canjeado (insertMany, con
bulkOperation: true)
    |
    ▼
Emitir un evento Socket.IO 'ticket-updated' por cada ticket actualizado
    |
    ▼
200 { processed, updated, alreadyRedeemed, total, tickets }

```

4. Decisiones de implementación relevantes

- **Idempotencia parcial en canje masivo:** si dentro de la selección hay tickets ya canjeados, la operación no falla por completo; simplemente los excluye y reporta cuántos se procesaron y cuántos ya estaban canjeados. Esto evita que un Jefe deba "limpiar" manualmente su selección antes de reintentar.
- **Auditoría no bloqueante:** un fallo al escribir el log de auditoría se captura en un bloque `try/catch` separado y **no revierte** el canje ya guardado; se prioriza que la operación de negocio (entregar la entrada) no se pierda por un problema secundario de logging.
- **bulkWrite en vez de N operaciones save() secuenciales:** para lotes grandes (compras corporativas), reduce drásticamente la cantidad de round-trips a la base de datos.
- **Notificación en tiempo real por ticket individual:** incluso en el canje masivo, cada ticket emite su propio evento `ticket-updated`, para que otros usuarios vean los boletos pintarse de verde uno por uno en la interfaz, en vez de una actualización masiva abrupta.

5. Alineación con historias de usuario

Historia de usuario	Cobertura
HU-03 — Canje individual de boleto	✓ Implementada (<code>canjeTicket</code>)
HU-04 — Canje masivo de boletos	✓ Implementada (<code>bulkCanjeTickets</code> , restringida a Jefe desde el fix de la Semana 4)
HU-09 — Actualización en tiempo real	✓ Implementada (emisión Socket.IO en ambos flujos)

6. Conclusiones del día

El proceso de canje queda completamente implementado, tanto en su variante individual como masiva, cumpliendo las reglas de negocio definidas desde la Semana 1 (bloqueo de doble canje, datos obligatorios de quien retira) y conservando trazabilidad y tiempo real en ambos casos.

Observaciones: Sin observaciones.